claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf_5de56b64-647\agent-aefc0be1dd951d3e6.jsonl`
- SHA-256: `1a7a9f8539aac499f739e7f2acf7558c6a6d4f455561b5b78e0ec6c1e54e97d0`
- Source modified: `2026-06-21T14:39:02+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_5de56b64-647`
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`

Transcript

1. user (2026-06-21T14:36:44.492Z)

Repo: C:\\Users\\avidu\\Projects\\badminton-highlight-indexer. Scope the M12 gdrive-api OAuth StorageBackend work item precisely.
Read backend/storage/** (esp. __init__.py, ref.py, base.py StorageBackend ABC, gcs.py as the DI template, and any gdrive.py mount-only backend), and the StorageConfig in backend/config/**.
Determine and cite file:line:
- The plan: backend/storage/gdrive_api.py GDriveApiStorage(StorageBackend) + a 'gdrive-api://' scheme. Confirm the scheme regex in ref.py already allows the hyphen; identify the exact edits to _KNOWN, _backend_class, and the StorageConfig Literal.
- How gcs.py does dependency-injection of its client (so M12 can mirror it with an INJECTED fake Drive for hermetic tests). Cite the constructor/factory.
- Whether the existing mount-only 'gdrive://' must stay untouched (confirm it exists + how it differs). Cite.
- What is claude-doable-no-spend (the backend + scheme + DI fake + tests) vs owner-gated (a real Google OAuth app / client secret / live Drive).
- DB migration: does M12 need one? Confirm current PRAGMA user_version and coordinate with M11's v7. Cite backend/infrastructure/database.py.
Give a realistic effort + the cleanest isolated-PR shape.

2. assistant (2026-06-21T14:36:46.859Z)

I'll analyze the badminton-highlight-indexer repository to scope the M12 gdrive-api OAuth StorageBackend work. Let me start by exploring the backend structure.

3. user (2026-06-21T14:36:47.571Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gcs.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\s3.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\ref.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage__init__.py

4. user (2026-06-21T14:36:47.896Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\presets.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py

5. user (2026-06-21T14:36:48.349Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\execution_policy.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py

6. assistant (2026-06-21T14:36:50.983Z)

Now let me read the critical files to understand the storage architecture:

7. user (2026-06-21T14:36:51.963Z)

```
1       """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3       Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4       live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5       pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6       no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7       """
8       from __future__ import annotations
9
10      import os
11      from abc import ABC, abstractmethod
12      from typing import Iterator
13
14      from backend.storage.ref import StorageRef, StorageStat
15
16
17      class StorageBackend(ABC):
18          """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19          worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21          @abstractmethod
22          def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24              """Make the object available as a local file; return its local path. For the
25              ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27          @abstractmethod
28          def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29                  content_type: "str | None" = None) -> StorageRef:
30              """Upload/copy a local file to ``ref``; return ``ref``."""
31
32          @abstractmethod
33          def exists(self, ref: StorageRef) -> bool: ...
34
35          @abstractmethod
36          def stat(self, ref: StorageRef) -> StorageStat: ...
37
38          @abstractmethod
39          def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40              """Yield a ``StorageRef`` per object under ``prefix``."""
41
42          @abstractmethod
43          def delete(self, ref: StorageRef) -> None: ...
44
45          def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46              """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47              sign (``local`` cannot; Drive only emulates) ? callers must treat
48              ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49              raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50
```

8. user (2026-06-21T14:36:52.330Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
```

```
57              return StorageRef("local", "", s, f"local://{s}")
58          scheme = m.group(1).lower()
59          rest = s[m.end():]
60          if scheme == "gs":
61              scheme = "gcs"
62          if scheme in ("local", "gdrive"):
63              # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64              return StorageRef(scheme, "", rest, s)
65          if scheme in _KNOWN:
66              bucket, _, key = rest.partition("/")
67              return StorageRef(scheme, bucket, key, s)
68          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
69
70
71      def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74          Precedence (first match wins):
75            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
78               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
79            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80                ``"gcs://bucket/x"``).
81            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
84          path used today, preserving byte-for-byte behaviour.
85          """
86          s = str(raw)
87          if _SCHEME_RE.match(s):           # 1. explicit scheme wins
88              return s
89          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93              return s
94          if input_root:                    # 3. join a relative name under the configured
root
95              return input_root.rstrip("/") + "/" + s.lstrip("/")
96          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
97              return f"{input_backend}://{s}"
98          return s                          # 5. bare/relative local path ? unchanged
99
```

9. user (2026-06-21T14:36:52.945Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
```

```
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13      from __future__ import annotations
14
15      import os
16      import tempfile
17      from typing import Any, Dict, Iterator, Optional
18
19      from backend.storage.base import StorageBackend
20      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22      __all__ = [
23          "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24          "get_storage", "fetch", "put", "list_uris",
25          "acquire_local_input", "input_is_local", "resolve_input_ref",
26      ]
27
28
29      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31          if storage_config is None:
32              return {}
33          if hasattr(storage_config, "model_dump"):
34              return storage_config.model_dump()  # type: ignore[no-any-return]
35          return dict(storage_config)
36
37
38      def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39          """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40          config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41          callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43          Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44          ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45          sc = _storage_dict(storage_config)
46          uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                                  input_root=sc.get("input_root", ""))
48          return parse_uri(uri)
49
50
51      def input_is_local(raw: str, storage_config: Any = None) -> bool:
52          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54          that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55          ``resolve_input_ref``)."""
56          return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59      def acquire_local_input(raw: str, storage_config: Any = None, *,
60                              scratch_parent: "str | None" = None) -> str:
61          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
```

```
processing.
62
63          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66          path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67          regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68          worker's ``cmd_infer`` already ships."""
69          sc = _storage_dict(storage_config)
70          ref = resolve_input_ref(raw, sc)
71          if ref.backend == "local":
72              return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
73          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74          return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77      def _backend_class(name: str):
78          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79          if name == "local":
80              from backend.storage.local import LocalStorage
81              return LocalStorage
82          if name == "s3":
83              from backend.storage.s3 import S3Storage
84              return S3Storage
85          if name == "gcs":
86              from backend.storage.gcs import GCSStorage
87              return GCSStorage
88          if name == "gdrive":
89              from backend.storage.gdrive import GDriveStorage
90              return GDriveStorage
91          raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94      def get_storage(backend: Optional[str] = None, *,
95                      config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96          """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97          the implementation; its per-backend settings come from ``config[backend]`` merged
with
98          ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99          cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107               config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
```

```
    def put(local_path: "str | os.PathLike[str]", uri: str, *,
            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
        """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
        ref = parse_uri(uri)
        get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
        return uri


    def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
        """Yield the URI of every object under ``prefix_uri``."""
        ref = parse_uri(prefix_uri)
        for r in get_storage(ref.backend, config=config).list(ref):
            yield r.uri

```

10. user (2026-06-21T14:36:53.485Z)

```
      """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.

      We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
Drive
      for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
requirement ? it
      needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
access. So
      ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
      your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
code the
      ``local`` backend uses).

          gdrive://Sharing/clip.mp4   ->   <mount>/My Drive/Sharing/clip.mp4

      Why an alias instead of just a raw path? Portability: the mount root differs per machine
      (Windows ``G:\\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
Drive``), so the same
      ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
      explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).

      A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
      ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
      talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
      fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
      """
      from __future__ import annotations

      import glob
      import os
      import string
      from typing import Iterator, Optional

      from backend.storage.base import StorageBackend
      from backend.storage.local import LocalStorage
      from backend.storage.ref import StorageRef, StorageStat

      _INSTALL_URL = "https://www.google.com/drive/download/"
```

```

    # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams files,
    # but a folder only appears once it's part of your synced Drive (and a *shared* folder must first
    # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but missing".
    _SYNC_REMEDY = (
        "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in Google "
        "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open Drive in the "
        "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'. Drive then "
        "downloads the bytes on first access."
    )


    def _not_mounted_msg(tried: Optional[str]) -> str:
        return (
            "Google Drive is not mounted, so the gdrive:// backend cannot resolve any path.\n"
            f"  Remedy:\n"
            f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
            f"  2. Then {_SYNC_REMEDY}\n"
            "  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive, macOS "
            "~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
            f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
        )


    def _not_synced_msg(root: str, rel: str, uri: str) -> str:
        return (
            f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
            f"(expected at {os.path.join(root, *rel.split('/'))!r}).\n"
            f"  This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
            f"  (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
        )


    def _autodetect_mount_root() -> Optional[str]:
        """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google Drive for
        Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and Windows
        ``<letter>:\\My Drive`` (``G:`` is the install default). Linux/other ? configure explicitly."""
        home = os.path.expanduser("~")
        for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage", "GoogleDrive*", "My Drive"))):
            if os.path.isdir(p):
                return p
        if os.name == "nt":
            # G is the Drive-for-Desktop default; then scan the rest of the assignable letters.
            for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
                p = f"{letter}:\\My Drive"
                if os.path.isdir(p):
                    return p
        return None


    class GDriveStorage(StorageBackend):
        def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
```

```
86              root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
_autodetect_mount_root()
87              if not root or not os.path.isdir(root):
88                  raise RuntimeError(_not_mounted_msg(root))
89              self._root = os.path.abspath(root)
90              self._local = LocalStorage(base_dir=self._root)  # all I/O is plain local-FS
under the mount
91
92          def _local_ref(self, ref: StorageRef) -> StorageRef:
93              """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
relative to
94              the mount root (``LocalStorage`` joins it under ``base_dir``)."""
95              rel = ref.key.lstrip("/")
96              key = os.path.join(*rel.split("/")) if rel else ""
97              return StorageRef("local", "", key, f"local://{key}")
98
99          def _require_present(self, ref: StorageRef) -> None:
100             """Guard the read paths: a path missing from the mount almost always means the
folder
101             isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
downstream."""
102             rel = ref.key.lstrip("/")
103             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
104             if not os.path.exists(abs_path):
105                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
106
107         def _to_gdrive_uri(self, abs_path: str) -> str:
108             """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
109             rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
110             return f"gdrive://{rel}"
111
112         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
113             # The file is already local (via the mount): no dst => identity passthrough to
the mount
114             # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
115             self._require_present(ref)
116             return self._local.fetch_to_local(self._local_ref(ref), dst,
overwrite=overwrite)
117
118         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
119             self._local.put(local_path, self._local_ref(ref), content_type=content_type)
120             return ref
121
122         def exists(self, ref) -> bool:
123             return self._local.exists(self._local_ref(ref))
124
125         def stat(self, ref) -> StorageStat:
126             self._require_present(ref)
127             return self._local.stat(self._local_ref(ref))
128
129         def list(self, prefix) -> Iterator[StorageRef]:
130             for r in self._local.list(self._local_ref(prefix)):
131                 uri = self._to_gdrive_uri(r.key)
132                 yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)
133
134         def delete(self, ref) -> None:
135             self._local.delete(self._local_ref(ref))
136
```

11. assistant (2026-06-21T14:36:55.952Z)

Now let me read the GCS backend to see the DI pattern, and the config models:

12. user (2026-06-21T14:36:56.881Z)

```
      """Google Cloud Storage backend ? first-class object buckets.

      Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
      install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
      (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
      never from config. A ``client`` may be injected for tests.
      """
      from __future__ import annotations

      from datetime import timedelta
      from typing import Any, Iterator, Optional

      from backend.storage.base import StorageBackend
      from backend.storage.ref import StorageRef, StorageStat


      class GCSStorage(StorageBackend):
          def __init__(self, *, client: Any = None, project: Optional[str] = None,
                       signed_url_ttl: int = 3600, **_ignored) -> None:
              self._ttl = signed_url_ttl
              if client is not None:
                  self._client = client
                  return
              from google.cloud import storage  # type: ignore  # lazy: only when a gcs:// URI
is used
              self._client = storage.Client(project=project)

          def _blob(self, ref: StorageRef):
              return self._client.bucket(ref.bucket).blob(ref.key)

          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
              if dst is None:
                  raise ValueError("GCSStorage.fetch_to_local requires a dst path")
              import os
              dst = str(dst)
              if os.path.exists(dst) and not overwrite:
                  return dst
              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
              self._blob(ref).download_to_filename(dst)
              return dst

          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
              self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
              return ref

          def exists(self, ref) -> bool:
              return bool(self._blob(ref).exists())

          def stat(self, ref) -> StorageStat:
              b = self._blob(ref)
              b.reload()
              return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
                                 content_type=b.content_type)

          def list(self, prefix) -> Iterator[StorageRef]:
              for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
                  yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")

          def delete(self, ref) -> None:
              self._blob(ref).delete()

```

```
61          def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62              return self._blob(ref).generate_signed_url(
63                  version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64
```

13. user (2026-06-21T14:36:57.777Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
```

```
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
73          # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
74          device: str = "cuda"        # auto | cuda | mps | cpu
75          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
76          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
77          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
78          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
79          keep_frames: bool = False
80          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
81          min_free_gb: float = 0.0
82          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
83          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
84          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
85          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
86          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
87          # Tri-state:
88          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
89          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
90          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
91          stream_video: Optional[bool] = None
92
93
94      class FusionConfig(BaseModel):
95          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
96
97          Every default reproduces control behaviour: the fusion segmenter persists the
98          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
99          true ? the person-cue features, but it does NOT gate the served handoff unless a
100         threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
101         is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
102         supplied ? off-court persons (spectators / adjacent courts) are excluded.
103         """
```

```
104         # Which trajectory substrate seeds the windows (shuttle stays primary).
105         substrate: Literal["wasb", "tracknet"] = "wasb"
106         # Windowing velocity threshold for the fusion family (the engine reads
107         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
108         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
109         velocity_threshold: float = 0.02
110         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
111         # enabled ? so the default path never imports torch / touches the GPU.
112         cue_flags: dict[str, bool] = Field(default_factory=dict)
113         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
114         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
115         min_crossings: int = Field(default=0, ge=0)
116         # Served Gate B: when the person cue is on, drop windows with median in-court
players
117         # < this. 0 = OFF.
118         min_players: int = Field(default=0, ge=0)
119         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
120         court_polygon: Optional[list[list[float]]] = None
121         # Net line for the person two-sided-motion split (person features only ? the shuttle
122         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
123         net_axis: Literal["x", "y"] = "y"
124         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
125
126
127     class IndexingConfig(BaseModel):
128         default_segmenter: str = "yolo_hybrid"
129         use_gpu: bool = True
130         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
131         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
132         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
133         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
134         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
135         detector_impl: str = "auto"
136         motion_threshold: float = 0.08
137         min_segment_duration: float = 3.0
138         dead_time_merge_gap: float = 2.0
139         chunk_duration: float = 90.0
140         chunk_overlap: float = 10.0
141         detector_model: str = "fasterrcnn_resnet50_fpn"
142         detector_score_threshold: float = 0.5
143         yolo_velocity_threshold: float = 0.15
144         yolo_frame_skip: int = 3
145         yolo_tracker: str = "bytetrack.yaml"
146         yolo_prefetch_workers: int = 2
147         min_action_window_duration: float = 2.0
148         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
149         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
150         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
151         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
152         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
153         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
154         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
155         max_window_duration: float = 6.0
156         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
```

```
157         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
158         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
159         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
160         # Only cuts (recall can't drop). OFF by default until measured/promoted.
161         window_hard_cap: bool = False
162         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
163         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
164         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
165         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
166         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
167         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
168         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
169         window_inactivity_end: float = 1.5
170         inactivity_rally_thresh: float = 0.20
171         inactivity_min_density: float = 0.30
172         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
173         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
174         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
175         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
176         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
177         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
178         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
179         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
180         window_blob_fallback: bool = False
181         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
182         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
183         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
184         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
185         window_blob_factor: float = 4.0
186         log_yolo_candidates: bool = True
187         store_yolo_candidates: bool = True
188         ai_handoff_padding: float = 2.0
189         ai_max_workers: int = 5
190         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
191         # chunks of a high-bitrate (4K) source blow past the provider upload cap
192         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
193         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
194         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
195         # Matches gemini_refine's --clip-scale-width default.
196         ai_chunk_scale_width: int = 1280
197         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
198         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
199         # emit their local candidate windows AS the rallies ? no provider, no API key,
```

```
nothing
200         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
201         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
202         skip_ai_handoff: bool = False
203         # Optional debug knob: trim every video to the first N seconds before processing.
204         debug_duration: Optional[float] = None
205
206         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
207         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
208         fusion: FusionConfig = Field(default_factory=FusionConfig)
209
210         @field_validator("default_segmenter")
211         @classmethod
212         def segmenter_must_be_registered(cls, v: str) -> str:
213             # Registry check happens at runtime in main/segmenter selection.
214             # Here we just allow any string for forward compatibility.
215             return v
216
217         @model_validator(mode="after")
218         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
219             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
220             # step makes `while t < duration: t += step` loop forever, growing memory before
any
221             # GPU work. Reject the bad config at load time instead.
222             if self.chunk_overlap >= self.chunk_duration:
223                 raise ValueError(
224                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
225                     f"({self.chunk_duration}); otherwise chunking never advances."
226                 )
227             return self
228
229
230     class OutputConfig(BaseModel):
231         default_highlights_name: str = "highlights.mp4"
232         db_name: str = "sports_indexer.db"
233         # Directory where the DB, highlight reels, and processed clips are written.
234         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
235         output_dir: str = "output"
236
237
238     class WatermarkConfig(BaseModel):
239         """Branding overlay burned into each highlight clip during the per-clip re-encode
240         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
241         (under `stitching.watermark` in config.json) to turn it off. The text is fully
242         configurable. The concat stage stays stream-copy (-c copy)."""
243         enabled: bool = True
244         text: str = "Generated by Khelsutra.guru"
245         logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
246         font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
247         font: str = "Sans"         # fontconfig family used when font_path is empty
248         font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
249         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
250         box: bool = True           # faint backing box behind the text for legibility
251         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
252         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"
253
254
255     class StitchingConfig(BaseModel):
```

```
256         begin_padding: float = 0.5
257         end_padding: float = 0.5
258         use_cache: bool = False
259         min_shot_count: int = 1
260         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
261         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
262         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
263         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
264         # local detector over-fires and its false positives are mostly SHORT (motion blips,
265         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
266         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
267         min_rally_duration: float = Field(default=0.0, ge=0.0)
268         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
269
270
271     class LoggingConfig(BaseModel):
272         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
273         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
274         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
275         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
276         # them to WARNING; set true to restore full chatter for request-flow debugging.
277         verbose_http: bool = False
278
279
280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = ""        # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = ""         # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
```

```
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330         input_root: str = ""
331         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
333         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335         # the legacy flat layout is used unchanged.
336         single_folder_per_video: bool = False
337         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338         workspace_root: str = ""
339         # Cloud namespace prefix under the root so per-video folders sit tidily beside
340         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341         workspace_prefix: str = "workspaces"
342         local: dict[str, Any] = Field(default_factory=dict)
343         s3: dict[str, Any] = Field(default_factory=dict)
344         gcs: dict[str, Any] = Field(default_factory=dict)
345         gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348     class DeploymentConfig(BaseModel):
349         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351
352         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
```

```
353         behaves exactly as before this section existed. A profile is opt-in (``config.json``
354         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375         inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
379         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
380
381         enabled: bool = False
382         pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to
a real one
383         fx_inr_per_usd: float = 83.0                  # USD?INR display rate (Bangalore
market)
384         margin: float = 0.0                            # resale markup (?0); 0 = bill at
cost
385
386
387     class AppConfig(BaseModel):
388         """Root configuration object.
389
390         All runtime configuration should be accessed via an instance of this class
391         (or the loader) rather than raw dict lookups.
392         """
393
394         sport: Sport = "badminton"
395         ai_provider: str = "gemini"
396         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
397         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
398         ai_model: str = "gemini-flash-latest"
399
400         validation: ValidationConfig = Field(default_factory=ValidationConfig)
401         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
```

```
402         output: OutputConfig = Field(default_factory=OutputConfig)
403         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
404         security: SecurityConfig = Field(default_factory=SecurityConfig)
405         logging: LoggingConfig = Field(default_factory=LoggingConfig)
406         storage: StorageConfig = Field(default_factory=StorageConfig)
407         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
408         billing: BillingConfig = Field(default_factory=BillingConfig)
409
410         # Allow extra keys during transition so existing config.json files do not break.
411         model_config = {
412             "extra": "allow",
413             "validate_assignment": True,
414         }
415
416         # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
417         # Will be removed once all call sites (validators, segmenters, etc.) move to typed
418         # attribute access / the typed config. A nested *section* is returned as a plain
dict
419         # via model_dump, so legacy chained access keeps working ? returning the Pydantic
420         # section model instead breaks it (a model has no .get), which bit every validator
on
421         # real runs. Scalar leaves are returned as-is.
422         def get(self, key: str, default: Any = None) -> Any:
423             _MISSING = object()
424             val: Any = _MISSING
425             if key in type(self).model_fields or hasattr(self, key):
426                 val = getattr(self, key, _MISSING)
427             if val is _MISSING:
428                 # Legacy convenience: find a leaf key inside any nested section. Sections
are
429                 # discovered dynamically from the model fields, so a future top-level
section
430                 # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
431                 for fname in type(self).model_fields:
432                     section = getattr(self, fname, None)
433                     if isinstance(section, BaseModel) and hasattr(section, key):
434                         val = getattr(section, key)
435                         break
436             if val is _MISSING:
437                 return default
438             if isinstance(val, BaseModel):
439                 return val.model_dump(mode="json")
440             return val
441
442         def __getitem__(self, key: str) -> Any:
443             val = self.get(key)
444             if val is None and key not in type(self).model_fields:
445                 # Re-raise like a real dict for unknown top-level
446                 raise KeyError(key)
447             return val
448
449
450     # Convenience type alias for callers that want to annotate
451     Config = AppConfig
452
```

14. user (2026-06-21T14:36:58.569Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
```

```

    from backend import billing  # pure cost math (no backend imports ? no cycle)

    logger = logging.getLogger(__name__)

    class Database:
        def __init__(self, db_path: str):
            self.db_path = db_path
            self._init_db()

        @contextmanager
        def _connect(self):
            """Transaction scope that also CLOSES the connection.

            ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
            never closes, so every call leaked its connection to GC (lingering file
            handles on the WAL sidecars under Windows). Internal methods use this.
            """
            conn = self._get_connection()
            try:
                with conn:
                    yield conn
            finally:
                conn.close()

        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
            """Run ONE bool-returning single-statement write inside a connection scope.

            Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
            the bool-returning single-statement writers. ``error_msg`` is the caller's exact
            per-method message (already formatted with its own ids) ? logged verbatim on a
            swallowed write fault so each writer keeps its specific log text. Returns True
on
            success, False (after logging ``error_msg``) on any exception."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(sql, params)
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"{error_msg}: {e}")
                return False

        @staticmethod
        def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
            """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.

            A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
```

```
            transaction, so an interrupted v3/v4 block can leave the column added while
            ``user_version`` stays un-bumped ? and the next open would re-run the same
            ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
            future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
            ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
            ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
            """
            try:
                cursor.execute(ddl)
            except sqlite3.OperationalError as exc:
                if "duplicate column name" not in str(exc).lower():
                    raise

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                self._create_base_tables(cursor)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    self._migrate_to_v1(cursor)

                if version < 2:
                    self._migrate_to_v2(cursor)

                if version < 3:
                    self._migrate_to_v3(cursor)

                if version < 4:
                    self._migrate_to_v4(cursor)

                if version < 5:
                    self._migrate_to_v5(cursor)

                if version < 6:
                    self._migrate_to_v6(cursor)
                # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 7

                conn.commit()

        def _create_base_tables(self, cursor):
            """Create the videos / play_segments / events base tables (idempotent)."""
            # Videos Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    id TEXT PRIMARY KEY,
                    filepath TEXT NOT NULL,
                    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    status TEXT NOT NULL,
                    validation_results TEXT
                )
            """)

            # Play Segments Table (Extensible metadata JSON)
            cursor.execute("""
```

```
                CREATE TABLE IF NOT EXISTS play_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    server TEXT,
                    receiver TEXT,
                    metadata TEXT,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                )
            """)

            # Events Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS events (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    segment_id INTEGER NOT NULL,
                    timestamp REAL NOT NULL,
                    type TEXT NOT NULL,
                    player TEXT,
                    details TEXT,
                    FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
                )
            """)

        def _migrate_to_v1(self, cursor):
            # v1: raw candidate detections (pre-confirmation), detector-agnostic.
            # Common fields are columns; detector-specific metrics go in `stats` JSON,
            # so a new segmenter reuses this table by setting its own `detector`/`stats`.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS candidate_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    detector TEXT NOT NULL,
                    chunk_index INTEGER,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    confidence REAL,
                    stats TEXT,
                    detection_params TEXT,
                    confirmed_segment_id INTEGER,
                    human_label TEXT,
                    notes TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
                )
            """)
            cursor.execute("""
                CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                    ON candidate_segments(video_id, detector)
            """)
            cursor.execute("PRAGMA user_version = 1")

        def _migrate_to_v2(self, cursor):
            # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
            # /api/process request. `status` is the EXECUTION state of the job
            # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
            # that failed validation) lives inside `result` JSON as result["status"].
            # `result` is the full sync-era response dict (incl. report paths), so the
            # observability report is the job's artifact, per the plan.
            cursor.execute("""
```

```
                CREATE TABLE IF NOT EXISTS jobs (
                    id TEXT PRIMARY KEY,
                    video_path TEXT NOT NULL,
                    video_id TEXT,
                    segmenter TEXT,
                    player_pool TEXT,
                    max_frames INTEGER,
                    status TEXT NOT NULL DEFAULT 'queued',
                    progress TEXT,
                    error TEXT,
                    result TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    started_at TIMESTAMP,
                    finished_at TIMESTAMP
                )
            """)
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
            cursor.execute("PRAGMA user_version = 2")

        def _migrate_to_v3(self, cursor):
            # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
            # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
            # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
            # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
            # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
            # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
            cursor.execute("PRAGMA user_version = 3")

        def _migrate_to_v4(self, cursor):
            # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
            # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
            # path can scope rows to an owner. **NULL = local install = byte-identical to
            # today** ? every existing row stays NULL and every existing query (which never
            # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
            # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
            # Additive ONLY: no served-behavior change rides on this slice.
            self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
            # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
            # of index churn while making the eventual per-user lookups cheap.
            cursor.execute(
                "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
                "WHERE user_id IS NOT NULL")
            cursor.execute(
                "CREATE INDEX IF NOT EXISTS idx_candidates_user "
                "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
            cursor.execute("PRAGMA user_version = 4")

        def _migrate_to_v5(self, cursor):
            # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
            # version): the resolved per-user `fusion_config` overlay + an INLINE
            # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
            # and provenance that earned it. `is_active` pins the one row the serving bridge
            # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
```

```
slice
250             # only persists/loads it ? NOTHING reads it on the served path yet.
251             #
252             #   user_id              ? owner of the model (NULL allowed = the local install)
253             #   version              ? monotonic per-user model version
254             #   baseline_version     ? the shared baseline this was fit against (upgrade
switch)
255             #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
256             #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
257             #   classifier_json      ? JSON LogisticRegression.to_dict() (None = config-only
model)
258             #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it was
promoted)
259             #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
260             #   is_active            ? the one resolved row for this user (partial-unique
below)
261             cursor.execute("""
262                 CREATE TABLE IF NOT EXISTS user_models (
263                     id INTEGER PRIMARY KEY AUTOINCREMENT,
264                     user_id TEXT,
265                     version INTEGER NOT NULL DEFAULT 1,
266                     baseline_version TEXT,
267                     feature_schema_hash TEXT,
268                     fusion_config TEXT,
269                     classifier_json TEXT,
270                     promotion_evidence TEXT,
271                     n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
272                     is_active INTEGER NOT NULL DEFAULT 0,
273                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
274                 )
275             """)
276             # One model version per user (NULLs compare distinct in SQLite, so the local
277             # install can still carry multiple versioned rows; the active-row guard below
278             # is what enforces a single served model).
279             cursor.execute("""
280                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
281                     ON user_models(user_id, version)
282             """)
283             # At most ONE active model per user: a partial-unique index over the active
rows.
284             # (SQLite treats NULL user_id rows as distinct, so the local install's single
285             # active row is still guarded ? there is one local user.)
286             cursor.execute("""
287                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
288                     ON user_models(user_id) WHERE is_active = 1
289             """)
290             cursor.execute("PRAGMA user_version = 5")
291
292         def _migrate_to_v6(self, cursor):
293             # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
294             # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
295             # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
296             # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
297             # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNs are
298             # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
299             for col, typ in (
```

```
300                 ("owner_id", "TEXT"),               # who pays (the non-owner user / tenant)
301                 ("compute_backend", "TEXT"),        # local_gpu | cloud_run | ...
302                 ("gpu_type", "TEXT"),               # L4 | T4 | ... (selects the per-second
GPU rate)
303                 ("gpu_active_seconds", "REAL"),
304                 ("wall_seconds", "REAL"),
305                 ("bytes_out", "INTEGER"),           # egress
306                 ("gemini_cost_usd", "REAL"),        # provider-reported, already USD
307                 ("pricebook_version", "TEXT"),
308                 ("cost_usd", "REAL"),               # computed total (USD = source of truth)
309                 ("cost_breakdown_json", "TEXT"),    # {gpu_seconds: .., egress_gb: ..,
gemini_usd: ..}
310             ):
311                 self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
{typ}")
312             # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
313             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314                            "WHERE owner_id IS NOT NULL")
315             # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
316             # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
317             cursor.execute("""
318                 CREATE TABLE IF NOT EXISTS pricebook (
319                     version TEXT NOT NULL,
320                     resource TEXT NOT NULL,
321                     gpu_type TEXT NOT NULL DEFAULT '',
322                     unit_price_usd REAL NOT NULL DEFAULT 0.0,
323                     currency TEXT NOT NULL DEFAULT 'USD',
324                     effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
325                 )
326             """)
327             cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
328                            "ON pricebook(version, resource, gpu_type)")
329             # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
(nothing
330             # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
$/GPU-s,
331             # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
seed.
332             for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
333                                        ("wall_seconds", ""), ("egress_gb", "")):
334                 cursor.execute(
335                     "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
unit_price_usd, "
336                     "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource,
gpu_type))
337             cursor.execute("PRAGMA user_version = 6")
338
339         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
340             """Register or update a video row WITHOUT touching its child segments.
341
342             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
343             with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
344             That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
345             before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
346             row in place; clearing segments is now an explicit, deliberate operation
347             (clear_video_data / prune_segments).
348             """
349             return self._execute_write(
```

```
                "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
                "ON CONFLICT(id) DO UPDATE SET "
                "filepath=excluded.filepath, status=excluded.status, "
                "validation_results=excluded.validation_results",
                (video_id, filepath, status, json.dumps(validation_results)),
                "Failed to add video",
            )

        def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
            """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).

            Captured before a (re)segmentation so the run's new rows (id > watermark) can be
            told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
            """
            with self._connect() as conn:
                cur = conn.cursor()
                p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
                                (video_id,)).fetchone()[0]
                return int(p), int(c)

        def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
                           play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
                           cand_after: Optional[int] = None) -> None:
            """Delete play/candidate segments by id range (events cascade from
play_segments).

            Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
            successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
            and keep the OLD ones when the run failed/produced nothing (``*_after``).
            """
            with self._connect() as conn:
                cur = conn.cursor()
                if play_upto is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
                if play_after is not None:
                    cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
                if cand_upto is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
                if cand_after is not None:
                    cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
                conn.commit()

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
```

```
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            return self._execute_write(
                "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
                (segment_id, timestamp, event_type, player, json.dumps(details or {})),
                "Failed to add event",
            )

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            return self._execute_write(
                "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
                (segment_id, candidate_id),
                f"Failed to link candidate {candidate_id} to segment {segment_id}",
            )

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"
```

```
457
458             query += " ORDER BY start_time"
459
460             candidates = []
461             with self._connect() as conn:
462                 rows = conn.cursor().execute(query, params).fetchall()
463                 for row in rows:
464                     d = dict(row)
465                     d["stats"] = json.loads(d["stats"] or "{}")
466                     d["detection_params"] = json.loads(d["detection_params"] or "{}")
467                     candidates.append(d)
468             return candidates
469
470         def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
471             with self._connect() as conn:
472                 row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
473                 if row:
474                     d = dict(row)
475                     d["validation_results"] = json.loads(d["validation_results"] or "[]")
476                     return d
477             return None
478
479         def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
480             """
481             Dynamically queries play segments based on filters:
482             - duration_min: float
483             - server: string
484             - receiver: string
485             - event_type: string (e.g. smash, foul)
486             """
487             query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
488             params = [video_id]
489
490             if filters.get("duration_min"):
491                 query += " AND r.duration >= ?"
492                 params.append(filters["duration_min"])
493
494             if filters.get("server"):
495                 query += " AND r.server = ?"
496                 params.append(filters["server"])
497
498             if filters.get("receiver"):
499                 query += " AND r.receiver = ?"
500                 params.append(filters["receiver"])
501
502             if filters.get("event_type"):
503                 # Check if there is an event of a specific type inside this segment
504                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
505                 params.append(filters["event_type"])
506
507             segments_list = []
508             with self._connect() as conn:
509                 cursor = conn.cursor()
510                 rows = cursor.execute(query, params).fetchall()
511
512                 for row in rows:
513                     r_dict = dict(row)
514                     r_id = r_dict["id"]
515                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
516
517                     # Fetch associated events
518                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
```

```
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) ------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None,
                       owner_id: Optional[str] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
            (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default)."""
            return self._execute_write(
                "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
                "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
                (job_id, video_path, segmenter,
                 json.dumps(player_pool) if player_pool is not None else None,
                 max_frames, json.dumps(policy) if policy is not None else None, owner_id),
                f"Failed to create job {job_id}",
            )

        def mark_job_running(self, job_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                "progress='starting' WHERE id = ?", (job_id,),
                f"Failed to mark job {job_id} running",
            )

        def set_job_progress(self, job_id: str, progress: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
                f"Failed to set progress for job {job_id}",
            )

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
                f"Failed to set video_id for job {job_id}",
            )

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            return self._execute_write(
```

```
                "UPDATE jobs SET status='done', progress='finished', result=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                (json.dumps(result), job_id),
                f"Failed to mark job {job_id} done",
            )

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            return self._execute_write(
                "UPDATE jobs SET status='failed', error=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
                f"Failed to mark job {job_id} failed",
            )

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
----------------------------
        def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) ->
Dict[str, float]:
            """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
            GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=''``.
            An unknown version ? ``{}`` (so every cost computes to 0.0)."""
            gt = gpu_type or ""
            rates: Dict[str, float] = {}
            with self._connect() as conn:
                rows = conn.cursor().execute(
                    "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version
= ?",
                    (version,)).fetchall()
            for r in rows:
                res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
                if res == billing.GPU_SECONDS:
                    if row_gt == gt:        # the GPU rate for THIS gpu_type
                        rates[res] = price
                else:
                    rates[res] = price      # wall/egress: gpu_type-independent
            return rates

        def record_job_cost(self, job_id: str, *, usage: Dict[str, float],
pricebook_version: str,
                            gpu_type: Optional[str] = None, compute_backend: Optional[str] =
None,
                            owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
            """Compute the job's cost from ``usage`` × the pricebook + persist every cost
column.
            Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
            ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
```

```
on."""
630             rates = self.get_pricebook_rates(pricebook_version, gpu_type)
631             # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
632             # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
633             # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
634             gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
635             unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
636             if unpriced_gpu:
637                 logger.warning(
638                     "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-
seconds billed "
639                     "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix
gpu_type.",
640                     gpu_type, pricebook_version, gpu_amt)
641             cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
642             if unpriced_gpu:
643                 breakdown["unpriced_gpu_seconds"] = gpu_amt
644             if margin:
645                 marked = billing.apply_margin(cost_usd, margin)
646                 breakdown["margin"] = round(marked - cost_usd, 6)   # keep ?(breakdown) ==
cost_usd
647                 cost_usd = marked
648             bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
649             ok = self._execute_write(
650                 "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
651                 "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
652                 "cost_breakdown_json=? WHERE id = ?",
653                 (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
654                  usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
655                  pricebook_version, cost_usd, json.dumps(breakdown), job_id),
656                 f"Failed to record cost for job {job_id}")
657             if not ok:
658                 return None
659             return {"cost_usd": cost_usd, "cost_breakdown": breakdown,
660                     "pricebook_version": pricebook_version}
661
662         def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
663             """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
``None`` until
664             recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
665             with self._connect() as conn:
666                 row = conn.cursor().execute(
667                     "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
668                     "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version, cost_usd,
"
669                     "cost_breakdown_json FROM jobs WHERE id = ?", (job_id,)).fetchone()
670             if not row:
671                 return None
672             d = dict(row)
673             raw = d.pop("cost_breakdown_json", None)
674             d["cost_breakdown"] = json.loads(raw) if raw else {}
675             return d
676
677         def get_video_cost(self, video_id: str) -> Dict[str, Any]:
678             """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
A video is
679             1:1 with an infer job, but a re-ingest can produce several ? sum them."""
680             with self._connect() as conn:
681                 rows = conn.cursor().execute(
```

```
682                     "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM jobs "
683                     "WHERE video_id = ?", (video_id,)).fetchall()
684             jobs: List[Dict[str, Any]] = []
685             total = 0.0
686             for r in rows:
687                 cost = r["cost_usd"]
688                 if cost is not None:
689                     total += float(cost)
690                 jobs.append({
691                     "job_id": r["id"], "cost_usd": cost, "pricebook_version":
r["pricebook_version"],
692                     "cost_breakdown": json.loads(r["cost_breakdown_json"]) if
r["cost_breakdown_json"] else {},
693                 })
694             return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
695
696         # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
697         def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
698             """Attach/replace a job's JSON ExecutionPolicy."""
699             return self._execute_write(
700                 "UPDATE jobs SET policy=? WHERE id = ?",
701                 (json.dumps(policy) if policy is not None else None, job_id),
702                 f"Failed to set policy for job {job_id}",
703             )
704
705         def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
706             """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
707             worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
708             computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
709             return self._execute_write(
710                 "UPDATE jobs SET status='waiting', "
711                 "next_poll_at=datetime('now', ?), "
712                 "progress=COALESCE(?, progress) WHERE id = ?",
713                 (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
714                 f"Failed to set job {job_id} waiting",
715             )
716
717         def seconds_until_next_due(self) -> Optional[float]:
718             """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
719             schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
720             already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
721             try:
722                 with self._connect() as conn:
723                     row = conn.cursor().execute(
724                         "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
725                         "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
726                         "FROM jobs WHERE status='waiting'").fetchone()
727                 if row is None or row["secs"] is None:
728                     return None
729                 return max(0.0, float(row["secs"]))
730             except Exception as e:
731                 logger.error(f"Failed to compute next-due delay: {e}")
732                 return None
733
734         def claim_due_job(self) -> Optional[Dict[str, Any]]:
735             """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
```

```
``next_poll_at`` is due ?
            flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
            double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
            first. Returns the claimed job dict, or None if nothing is runnable."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    row = cur.execute(
                        "SELECT id, status FROM jobs WHERE status='queued' "
                        "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
                        "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
                    if not row:
                        return None
                    cur.execute(
                        "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
                        "WHERE id=? AND status=?", (row["id"], row["status"]))
                    conn.commit()
                    if cur.rowcount != 1:
                        return None                     # lost the race to a concurrent
worker
                return self.get_job(row["id"])
            except Exception as e:
                logger.error(f"Failed to claim a due job: {e}")
                return None

        def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
            """Crash recovery: any job still queued/running from a previous process is dead
            (the executor is in-process). Mark failed so clients see a terminal state, not a
            hang. Called once at startup. Returns the number of jobs swept."""
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    cur.execute(
                        "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
                        "WHERE status IN ('queued', 'running')", (reason,))
                    conn.commit()
                    return cur.rowcount
            except Exception as e:
                logger.error(f"Failed to sweep stale jobs: {e}")
                return 0

        # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
        # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
        # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
        # serving.py) resolves them, and wiring that bridge into the production decision
seam
        # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
        # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
        # JSON-serialised dicts; the rest are scalar columns.

        def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
                              baseline_version: Optional[str] = None,
                              feature_schema_hash: Optional[str] = None,
                              fusion_config: Optional[Dict[str, Any]] = None,
                              classifier_json: Optional[Dict[str, Any]] = None,
                              promotion_evidence: Optional[Dict[str, Any]] = None,
                              n_videos_at_fit: int = 0,
                              is_active: bool = False) -> Optional[int]:
            """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.

```

```
            Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
            rowid). When ``is_active`` is True this row becomes the user's single active
model ?
            any previously-active row for the SAME user is first cleared, so the
            ``idx_user_models_one_active`` partial-unique invariant always holds (one active
model
            per user). Returns the row id, or None on failure.
            """
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if is_active:
                        # Demote the current active row first (NULL user_id = local: IS NULL
match).
                        if user_id is None:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
                                (version,))
                        else:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id = ? AND is_active = 1 AND version != ?",
                                (user_id, version))
                    cur.execute(
                        """INSERT INTO user_models
                           (user_id, version, baseline_version, feature_schema_hash,
                            fusion_config, classifier_json, promotion_evidence,
                            n_videos_at_fit, is_active)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                           ON CONFLICT(user_id, version) DO UPDATE SET
                             baseline_version=excluded.baseline_version,
                             feature_schema_hash=excluded.feature_schema_hash,
                             fusion_config=excluded.fusion_config,
                             classifier_json=excluded.classifier_json,
                             promotion_evidence=excluded.promotion_evidence,
                             n_videos_at_fit=excluded.n_videos_at_fit,
                             is_active=excluded.is_active""",
                        (user_id, version, baseline_version, feature_schema_hash,
                         json.dumps(fusion_config) if fusion_config is not None else None,
                         json.dumps(classifier_json) if classifier_json is not None else
None,
                         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
                         n_videos_at_fit, 1 if is_active else 0))
                    conn.commit()
                    rid = cur.execute(
                        "SELECT id FROM user_models WHERE "
                        + ("user_id IS NULL" if user_id is None else "user_id = ?")
                        + " AND version = ?",
                        ((version,) if user_id is None else (user_id, version))).fetchone()
                    return int(rid[0]) if rid else None
            except Exception as e:
                logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
                return None

        def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
            """Return the single active per-user model row for ``user_id`` (NULL = local
install),
            with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
            or None if the user has no active model. Does not touch the served pipeline."""
            with self._connect() as conn:
```

```
849                 if user_id is None:
850                     row = conn.cursor().execute(
851                         "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
852                 else:
853                     row = conn.cursor().execute(
854                         "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
855                         (user_id,)).fetchone()
856                 if not row:
857                     return None
858                 d = dict(row)
859                 for col in ("fusion_config", "classifier_json", "promotion_evidence"):
860                     d[col] = json.loads(d[col]) if d[col] else None
861                 return d
862
```

15. assistant (2026-06-21T14:37:01.175Z)

Let me also check the S3 backend for DI comparison and look for any existing gdrive-api references:

16. user (2026-06-21T14:37:02.202Z)

```
1       """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3       One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4       only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5       ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7       Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8       style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9       / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10      (tests pass a fake; production lets the constructor build one via boto3).
11      """
12      from __future__ import annotations
13
14      import os
15      from typing import Any, Iterator, Optional
16
17      from backend.storage.base import StorageBackend
18      from backend.storage.ref import StorageRef, StorageStat
19
20
21      class S3Storage(StorageBackend):
22          def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                       region: Optional[str] = None, addressing_style: str = "auto",
24                       signed_url_ttl: int = 3600, **_ignored) -> None:
25              self._ttl = signed_url_ttl
26              if client is not None:
27                  self._client = client
28                  return
29              import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
30              from botocore.config import Config  # type: ignore
31              self._client = boto3.client(
32                  "s3",
33                  endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34                  region_name=region,
35                  config=Config(s3={"addressing_style": addressing_style}),
36              )
37
38          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39              if dst is None:
```

```
40                  raise ValueError("S3Storage.fetch_to_local requires a dst path")
41              dst = str(dst)
42              if os.path.exists(dst) and not overwrite:
43                  return dst
44              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
45              self._client.download_file(ref.bucket, ref.key, dst)
46              return dst
47
48          def put(self, local_path, ref, *, content_type=None) -> StorageRef:
49              extra = {"ContentType": content_type} if content_type else None
50              self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
51              return ref
52
53          def exists(self, ref) -> bool:
54              try:
55                  self._client.head_object(Bucket=ref.bucket, Key=ref.key)
56                  return True
57              except Exception:  # noqa: BLE001 - any boto/client error means "not found /
unreachable"
58                  return False
59
60          def stat(self, ref) -> StorageStat:
61              h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
62              return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
63                                 etag=h.get("ETag"), content_type=h.get("ContentType"))
64
65          def list(self, prefix) -> Iterator[StorageRef]:
66              paginator = self._client.get_paginator("list_objects_v2")
67              for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
68                  for obj in page.get("Contents", []) or []:
69                      key = obj["Key"]
70                      yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")
71
72          def delete(self, ref) -> None:
73              self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
74
75          def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
76              op = "get_object" if method.upper() == "GET" else "put_object"
77              return self._client.generate_presigned_url(
78                  op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)
79
```

17. user (2026-06-21T14:37:03.248Z)

```
Found 4 files
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\NEXT_STEPS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DOC_STATUS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\decisions\DECISIONS.md
```

18. assistant (2026-06-21T14:37:05.169Z)

Now let me check the config loader and look at M11/M12 tracking in docs:

19. user (2026-06-21T14:37:06.202Z)

```
Exit code 1
/usr/bin/bash: line 1: cd: C:UsersaviduProjectsbadminton-highlight-indexer: No such file or
directory
```

20. user (2026-06-21T14:37:09.329Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/.pytest_cache/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/assets/fonts/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-eval-baseline/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-logging-reporting/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-scaffolding-complete/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-strategy-foundation/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/decisions/DECISIONS.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/past_projects/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-first-ab-experiment/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-human-gt-tuning/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-initial/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/2026-06-multivideo-lovo/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/quality-iterations/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/NEXT_STEPS.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/scripts/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/training/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/.pytest_cache/README.md

21. user (2026-06-21T14:37:12.135Z)

1 # Next steps (live) ? refreshed 2026-06-21
2
3 > **How to read:** this top section is the **current cross-track state + prioritized pending list** (refreshed 2026-06-21, replacing the stale 2026-06-13 header). The detailed **owned-model & rally-quality track** plan (2026-06-13) is **preserved below, still valid** ? it is now *one of several* active tracks. Live blow-by-blow: memory `current-state.md`. Doc lifecycle/hygiene: `docs/DOC_STATUS.md`.
4
5 ## ? Where we are (2026-06-21)
6 **DECOUPLED_COMPUTE delivered on GCP + archived** (#270); **compute = GCP only** (ADR-013, DO dropped). **`COMPUTE_DECOUPLED_SERVING`** subproject spun up (#272, **ADR-014**) ? one pure core + local/cloud profiles; **owner M0 design sign-off pending**. **Per-video workspace P0** shipped (#264); **doc-status register + archival due-diligence** in place (#273). The **khelsutra site + per-rally UI** ship in parallel (sibling track). The **rally-quality /

owned-model** track (below) is the core ML work, gated on **growing the golden corpus**.

? Prioritized pending ? all tracks

P0 ? highest leverage / unblocks the most

0. **? [owner-priority, 2026-06-21] MAKE IT UI-DRIVEN + ALPHA-SHAREABLE ? "a few recordings lying around ? highlights, from a page, no SSH".** Today the cloud flow is **SSH/CLI-only** (`docs/CLOUD_GPU_DRYRUN_GUIDE.md`); **M12 does NOT fix it.** Needs (a separate khelsutra `web/` SPA + deploy track, NOT an engine seam): **(i)** deploy the engine as a reachable **service** (M7 ? Cloud Run service, ? owner GCP spend for a *persistent* endpoint), **(ii)** the **SPA ingest/progress screen** (khelsutra `web/` B-P2: enter/upload video ? `/api/process` ? poll `/api/jobs` ? reel; ZERO engine code), **(iii)** flip **M9 edge-auth ON** (merged #290, default-OFF) to gate alpha tenants, **(iv)** a **compute PICKER in the UI** ("your machine vs our cloud", D13/D15). The engine seams make it *possible*; the SPA makes it *usable*. Full record: [[ui-driven-cloud-serving-gap]].
1. ? **`COMPUTE_DECOUPLED_SERVING` M0?M9 LARGELY SHIPPED** (2026-06-21, ADR-014 ratified). M1?M4 (#279/#280/#282/#283), **M5 #286, M6 #287, M8 #288, M9-auth #290** all merged (default-OFF); the **real M7 L4 run** validated the worker (22 rallies, VM deleted, $0). All §8 gating Qs locked (D7?D17). **Owner residual** (not blocking Claude's default-OFF code): counsel ToS/DPA (M9-consent live + M11 live), real GCP prices+FX (M8 calibrate), Google OAuth app (M12 real), CF team-domain/AUD + ingress-lock; real cloud verification authorized within a **$100/?10k cap** (D17).
2. **[owner, ongoing] Grow the golden corpus** ? multi-court badminton + ?3 matches/sport (TT first). The single highest-leverage move; the quality track *and* full-corpus training feed on it.
3. **[owner + Claude · GPU] Full-corpus training (n?5)** ? a real `weights/<ver>/` (the `0.1.0-l4` bundle is n=2 plumbing-only).

P1 ? near-term, mostly Claude-doable (default-OFF, green-gated)

4. **[Claude] `COMPUTE_DECOUPLED_SERVING` batch ? REMAINING: M11 + M12** (M5/M6/M8/M9-auth ? merged this session). **M11** = `backend/flywheel.py` capture?store?shadow-eval?goldenize plumbing reusing `workspace.py::for_video` + `backend/eval/experiment.compare` + `backend/eval/promotion.py::promote_if_served_safe`, behind `flywheel.*`=OFF + a faked `consent_attested` (? privacy-sensitive ? give it fresh focus). **M12** = `backend/storage/gdrive_api.py` `GDriveApiStorage(StorageBackend)` + a `gdrive-api://` scheme (regex already allows the hyphen; add to `_KNOWN` + `_backend_class` + the StorageConfig Literal) against an INJECTED fake Drive (mirror `gcs.py` DI); keep the mount-only `gdrive://` untouched. ? **M9-consent cols = a new v7 migration** (M8 took v6). M9-consent ToS/DPA text = owner/counsel.
5. **[Claude] Per-video workspace P1?P6** (P1 v6 DB migration ? P2 worker ? P3 per-video routing of `detector/`+`tmp/` *[the `output/chunks_*` de-hardcode base = ? done via #282]* ? P4 ? P5 Launch-Report flip ? P6 GPU parity). ? P1's v6 migration coordinates with the M8 v6 cost-ledger migration ? author the version bump once.
6. ? **Ops-Agent codified (#291)** ? `deploy/gcp/create_gpu_vm.sh` (the one-command VM helper) always bakes the Ops-Agent startup-script; provision.sh grants the APIs+SA roles. PROVEN active on a fresh VM 2026-06-21. ([[gcp-ops-agent-enable]])
7. **[owner] Set the ship-gate target (#172)** ? needs corpus growth.
8. **[owner-side GPU] R-series: promote-or-reject `R2` tolF1 as the gate** (the ablation) ? needs GPU-extracted golden features.

P2 ? later / gated / deferred

9. Rally-quality **P3/P4** (person-fusion LOVO, audio cue) ? needs GPU golden features.
10. **M0.4 scorer swap** ? TemporalMaxer (per the 2026-06-17 external review).
11. **Multisport** (table-tennis first, then pickleball).
12. Serving **M8/M9** ? per-video cost ledger, edge auth, DPA/consent (within `COMPUTE_DECOUPLED_SERVING`; owner/legal-gated).
13. **WASB-C3** weight provenance (gates *sharing* a trained model) ? legal.
14. **Rename repo off "badminton"** (owner, BACKLOG).
15. Owner housekeeping: retire the shared **DO droplet** + **DO token**; collector **`md5`** export (collector repo).
16. **PMF field validation** (owner-side, business track).

Doc-hygiene (tracked in `docs/DOC_STATUS.md` §4): ? #1 per-video reconcile + ? #2 this refresh; **pending:** field/PMF/Roora ? archives + a data-layer `docs/` dir (owner boundary confirm); confirm the COMPLETED top-level stubs; verify `HOSTING_PLAN`/`UI_PROPOSAL` currency.

Owned-model & rally-quality track ? detail (2026-06-13 plan; still valid)

Updated: 2026-06-13 (**rally-detection quality track**: fusion + experiment-harness design docs merged,
owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior: 2026-06-10 (audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-first delivery ? see
`docs/UI_ALPHA_EXECUTION_PLAN.md`; **PR-1 async jobs (#16) MERGED 2026-06-11, PR #87**; **PR-2
upload/download MERGED 2026-06-11, PR #99** ? live-E2E-verified (3-part chunked upload MD5 byte-identity +
compile?browser-download); next: PR-3 identity. Same day: the **8?9 quality sweep #90?#98 landed** ? ruff/mypy/
coverage-floor CI lanes, SQLite conn-close fix, config unknown-key warning, hybrid-twin collapse into
`trajectory_hybrid.py`, WSL tilde+abspath live-path fixes; suite 388 green; LOVO 0.626 re-verified exact).
Authoritative roadmap:
`docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+ ADR-008 for topology). **Latest blow-by-blow:** memory
`current-state.md` / `session-handoff.md`. This file = the prioritized "what to do next," refreshed each session.

? Next-session / GPU-box pickup (2026-06-15)

M0 in-container quick wins are COMPLETE: ship-gate provisional blocker (**#170**), train/eval split guard (**#171**), the **doc-consistency sweep** (**#174** ? corrected 16 drifted docs vs the code), and the **P3 person-feature fusion litmus machinery is already present + suite-green** (`backend/eval/fusion_compare.py` + `ablation.py`: person-driven ?F1 / bootstrap CI / paired sign test; the real-golden `skipif` litmus waits only on GPU-extracted data). ? CI is red repo-wide due to a **GitHub Actions minutes/billing outage** (account-level, not code) ? work is local-gated + `--admin`-merged until Actions is restored (Settings ? Billing ? Actions minutes / spending limit).

Immediate next step ? the real P3 LOVO measurement (this box has a GPU + `torch+cu124` + `cv2`; videos and trajectory CSVs are present, but the GPU-extracted golden features + `.rallies.csv` labels are not yet here):

1. **[owner / GPU box]** `python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-data\features\2026-06-15-n6"` ? produces `*_golden_features.json` for the 6 golden videos. (GPU + WSL + videos + trajectory CSVs + `.rallies.csv` labels are owner-side only.)
2. Artifacts land in the **shared cloud folder** `C:\Users\avidu\OneDrive\rally-annotator\golden-data\` (OneDrive auto-syncs both ways). Convention + workflow: **[`GOLDEN_DATA_SHARING.md`](data_pipeline/GOLDEN_DATA_SHARING.md)**; tracked in **#175**.
3. **[CPU dev/agent session]** `python -m backend.eval.fusion_compare --features-dir <shared>\features\2026-06-15-n6 --record` + `python -m backend.eval.ablation --features-dir <shared>\... --granularity group` ? real person-feature ?F1; copy the JSONs into `output/` to also pass the `skipif` litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`).

Also queued (owner-gated / pending input):

- Ship-gate target calibration ? **#172** (owner decision; needs corpus growth).
- Tests reorganization ? blocked on the owner's grouping metric (provide it ? an agent will do it).
- Restore CI ? the Actions outage is account-level (billing/minutes), not a repo/config bug.

Leaving (bulky / M2 ? no start without owner go): ActionFormer (M0.4), event-index DB migration (M0.1), Gemini-silver purge (M0.3), cost-to-Gen-2 benchmark + ByteTrack deprecation, multisport, repo rename, PMF execution.

Where we are (one paragraph)

ADR-008 ratified ("repo split driven by productionization, not isolation"): training lives in-repo (`training/`)

```
67      behind a CI-enforced import wall; the featurizer is single-sourced
(`backend/features/rally_seq.py`, train==serve);
68      the **Gen-0 harness ships** (`python -m training.gen0.harness --manifest
output/human_lovo_manifest.json --floor
69      eval_baselines/heuristic_lovo_n6.json --version <semver>`) and produced artifact v0.1.0
(LOVO **0.626**, floor passed,
70      sign test 5/6 wins p=0.109 ? honestly not significant, ship=False). The **n=6 owned
corpus is in the collector**
71      (262 rallies, Proprietary, commercial export = owned data only after the ShuttleSet
reclassification). The **Gemini
72      battery is done** (see table) ? the moat is quantified.
73
74      ## The quality table that now drives strategy (IoU 0.5 vs human golden labels)
75      | Path | Mahadevpura (clean-ish) | Kushagra (multi-court) |
76      |---|---|---|
77      | Heuristic (velocity windowing) | 0.657 | 0.157 |
78      | **Gen-0 owned head** (LOVO, n=6) | 0.744 | 0.561 |
79      | Two-stage WASB+Gemini refine | 0.83 | ? |
80      | **Gemini wrapper** (`gemini-flash-latest`) | **0.93** | **0.66** |
81
82      **Reading:** clean footage is commoditized (serve it with Gemini for cents). **Multi-
court drops the commodity
83      wrapper to 0.66 ? that is the owned model's ship target** (its FPs are background-game
pickups + dead-time merges,
84      the exact contrast-metric confound). Gen-0 is 0.10 behind with only n=6 ? a corpus-
growth-sized gap, not an
85      architecture-sized one. Two-stage refine = the cost-efficient serving path for LONG
tapes (uploads candidate clips
86      only); wrapper 503-flakiness (observed all session) makes the provider-fallback registry
load-bearing.
87      Baselines tracked in `eval_baselines/` (heuristic_lovo_n6.json,
gemini_wrapper_2026-06-10.json).
88
89      ## Rally-detection quality track (design merged 2026-06-13, owner-gated; implementation
in progress)
90      **This is the track a desktop/local agent is slated to pick up.** Design docs landed (PR
#112 + session). All implementation is **owner-gated** (per explicit in the plan); each step =
ONE PR off `master`.
91
92      **Hardening loop (T1-T5 positive tests + audit + finale) COMPLETE (2026-06-14, PRs
#156/#157/#160/#161/#163 + #165 + final records).** See archived
`docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` (Living Log with every
pre/post/review) and `docs/archives/HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md` (full rules
+ STATE). Top-level now pointer stubs per #165 review. All 5 items + verification + archive
done; clean slate.
93
94      - **Docs:** `docs/MULTI_SIGNAL_FUSION_PLAN.md` (shuttle + person + audio, one fusion
layer) ·
95        `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
96        `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md`
(2-layer mental model).
97      - **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a
rally" bug is **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already
implements the net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills
service-feed/held-shuttle windows, but it is **only called from eval drivers** (and there was no
`min_crossings` knob in `IndexingConfig`). **Wiring Gate A into the live segmenter (now via
fusion_hybrid) was the single cheapest, highest-confidence quality win.** (P2 FusionSegmenter
specifics: registry `fusion_hybrid`, **default-OFF**, seeds from substrate, persists
`net_crossings` + optional person features, optional served Gate A/B via config knobs;
adversarially reviewed; suite green. See `tests/test_served_gate_a_regression.py` for the honest
finding that on production windowing Gate A is neutral/negative in some cases, hence kept opt-in
with knob for safety, and the leaderboard revert note.)
98      - **Current status (2026-06-14):** P1 (person cue extraction to reusable
`PersonDetector`) + P2 (FusionSegmenter + Gate A signal + served handoff gating) + experiment
harness P1 DONE (adversarial review + NO-REGRESSION; suite green). **P3 next (this PR + follow-
```

ups):** learned fusion (person features ? harness `compare` LOVO lift on the 6 golden videos; eager-person-compute optimisation); then P4 audio via hit_detector.
99   - **Pick-up order (each = ONE PR off `master`, owner approval first, flag-gated default-OFF, promote only on measured LOVO):**
100   1. (P2 first-step, largely done via fusion_hybrid) Wire Gate A into production served path + config knob (min_crossings); apply in hybrids at runtime. (Now available opt-in; served neutral on production windowing per litmus ? kept opt-in with knob for safety.)
101   2. P3 first step (this work item): harness `compare` LOVO litmus for person-on variant (players_in_court + two_sided + colocation etc. from `fusion_features`) on the 6 golden; add the eager compute optimisation note/landing. Pure + harness (testable here).
102   3. P3/P4 continuation: audio cue integration + full learned promotion if lift proven.
103   - **Discipline (from EXPERIMENT_HARNESS.md):** every new cue **flag-gated, default OFF**; promote to default **only on measured LOVO lift** through `run_regression.py` (exit 0/1/3); pinned `CONTROL` ? rollback = config flip. Harness re-uses ~80% existing code.
104   - ? Gate A, person cue logic, harness, and pure fusion paths are testable here; the heavier real-video / golden LOVO / ablation confirmation waits on the GPU-extracted golden features + `.rallies.csv` labels, which are not yet present in this checkout.
105   - **External due-diligence review inputs (2026-06-17) ? PICK UP AFTER current ideas are exhausted; full details + citations in [`RALLY_QUALITY_RESEARCH.md`](RALLY_QUALITY_RESEARCH.md) §5.6.** An owner-commissioned 31-reference review of the quality report *validated* the methodology and surfaced these **future** levers (research inputs, not active work, not new claims; verify citations before any submission):
106   1. **"First-mile" paper framing (highest-value):** stroke-classifiers (BST/DSTA) + the big datasets (ShuttleSet ~36k strokes, Fine-Badminton ~27k actions) all **assume a pre-trimmed rally clip already exists** ? we solve the ignored prerequisite. Pitch the paper as *"the missing untrimmed-video preprocessing pipeline that lets BST/MonoTrack run on raw footage."*
107   2. **Ground R2 in Action-Spotting / Precise-Event-Spotting** (SoccerNet/TenniSet/T-DEED) ? turns R2 into a defensible publishable contribution.
108   3. **Add domain baselines for any submission:** MonoTrack + BST on ShuttleSet / Fine-Badminton (we have only heuristic + Gemini today).
109   4. **Prefer TemporalMaxer over ActionFormer** for the M0.4 scorer swap (parameter-free max-pool, ~2.8× fewer GMACs, small-N robust) ? see "M0.4 next increment" below.
110   5. **Court polygons ? pose ? the rally-END lever:** masking on-court players can resolve *occluded end-of-rally states* (the #1 remaining gap vs Gemini), not just spectator noise.
111   6. **Name the eval?serve lesson "pipeline distribution skew"** ? a citable paper "lesson."
112
113   **Discipline reminder:** owner approval before starting any owner-gated item; one PR per step; babysit the PR (poll/fix/reply) until merge observed, *then* advance.
114
115   ## Prioritized next steps
116   1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move; every lever below feeds on it.
117   Priority order for new videos: **(a) multi-court badminton** (the target distribution AND where the ship target
118   lives), **(b) ?3 matches per new sport ? table tennis first** (WASB transfers at F1 0.909; pickleball labels are
119   corpus-gold but not trainable until a clean MIT/Apache ball detector exists), (c) capture variety per
120   `VIDEO_RECORDING_GUIDELINES.md`; **adult sessions only for the training pool** (consent guardrail). Per video:
121   label ? `curate import-local --normalize --license Proprietary --fps <true fps>` ? re-run the Gen-0 harness.
122   The paired sign test needs n: at n=10 with 9 wins, p?0.011 ? significance is reachable this month.
123   2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480 (necessary), **multi-court
124   reference = wrapper 0.66** (the number that makes the owned model worth serving), clean-footage ceiling = 0.93
125   (not the target ? Gemini serves that tier). Suggested shape: "LOVO F1@tIoU0.5 ? 0.70 on multi-court folds with
126   paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
127   3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for a lightweight temporal head (MIT, minutes on the

```
128        2070S ? NOT the heavyweight Gen-2 stack) inside the existing harness; per-video
threshold calibration (the
129        LOVO-vs-oracle gap is visible per fold in the harness output). **Prefer
TemporalMaxer** (parameter-free
130        max-pool, ~2.8× fewer GMACs, small-N robust) over ActionFormer/ASFormer per the
2026-06-17 external review
131        (`RALLY_QUALITY_RESEARCH.md` §5.6) ? better fit for the cheap/local/fast mandate.
132     4. **Multisport thread:** ingest Roora pickleball/TT CSVs (`import-local --normalize`);
merged-prototype LOVO across
133        badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-
on-WASB serving stays gated
134        by C3 (provenance multiplies per sport).
135     5. **PMF validation (cheaper than any engineering month):** C13 academy interviews (+
the two added questions:
136        "multi-court footage nobody watches?" / "pay per-match to index it?"); camera pilot
at the warmest 2-3 academies
137        (consent at capture; demo served via the Gemini path or human-polish ? the reel
pattern exists:
138        `output/MahadevpuraSingles_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner
ratified the
139        physical-visit approach; reframed product-feedback-first per owner spec):**
140        `docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md` (the recruiting flyer ? owner fills ?/contact
placeholders and recruits) +
141        `docs/FIELD_VISIT_PLAYBOOK.md` (the hired associate's visit playbook: coach + student
script, demo-after-Q6
142        rule, ground rules, Phase-2 record-and-show-back stretch goal) +
`docs/FIELD_VISIT_ANSWER_SHEET.md` (per-visit
143        capture form incl. demo-reaction + student blocks) + `docs/PMF_FIELD_SIGNALS.md`
(G/A/R anchors + **pre-registered
144        decision rules** ? PROCEED / PIVOT A-C / FALSIFIED thresholds fixed before the first
visit).
145     6. **Housekeeping:** ~~rotate the Gemini API key~~ **DONE 2026-06-10** (owner-verified
working; confirm old-key
146        deletion in AI Studio); **rename repo off "badminton"** (owner action,
147        in BACKLOG); collector=SoR confirmation; off-peak niceties: none pending ? the
battery completed.
148
149     ## Pointers / artifacts
150     - Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Topology: ADR-008
(`docs/archives/decisions/DECISIONS.md`) ·
151       Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` · Quality history:
152       `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.
153     - Harness: `training/gen0/harness.py` (one command ? train ? LOVO ? gate ? artifact
bundle; weights gitignored,
154       manifest reviewable). Featurizer: `backend/features/rally_seq.py` (FEATURE_SCHEMA v1).
155     - Corpus: collector DB (system-of-record, 6 matches/262 rallies) · eval manifest
`output/human_lovo_manifest.json`
156       (gitignored) · durable trajectories + Roora files: `C:\Users\avidu\Projects\Annotation
Setup\Collect\Trajectories\` ·
157       labels: `...\Collect\Golden Labelled\`.
158     - Gemini ops learnings: serial uploads (`indexing.ai_max_workers=1`) + chunk re-encode
159       (`indexing.ai_chunk_scale_width=1280`) + 6-attempt exponential generate retry ? a
cold/prepaid project burst-throttles
160       with a MISLEADING "credits depleted" 429; ramp gradually.
161     - ? No owned-model implementation beyond greenlit M0.x until owner greenlights the next
milestone. Committed next
162       PLATFORM slice = async job model (single-tenant).
163
```

22. user (2026-06-21T14:37:12.803Z)

```
1       # Architecture Decision Records
2
3       ## ADR-001: Precise shuttle tracking via self-hosted TrackNetV4/WASB on WSL2
4
```

Date: 2026-05-25
Status: Accepted

Context

The pipeline uses an AI multimodal provider (Gemini) for *semantic* rally segmentation. We need **precise frame-level detection of the moving shuttlecock** (and, generically, balls for other sports) to produce higher-quality candidate rallies and a fine-tuning signal.

We evaluated whether a commercially available model/API could do this. Findings:

- **Multimodal LLM APIs (Gemini, Claude, GPT-class) are the wrong tool for precise localization.** They reason well about *what* is happening (rally vs. dead time) but cannot reliably track a tiny, fast (~up to 400 km/h), motion-blurred shuttle frame by frame.
- The precision leaders are **purpose-built trajectory models**, which are open source and self-hosted, not big-vendor APIs:
  - **TrackNetV4** (TensorFlow) ? badminton-specific, heatmap + motion-attention, highest reported shuttle accuracy; ships `src/predict.py` producing per-frame (X, Y) coordinate CSV. https://github.com/AR4152/TrackNetV4
  - **WASB** (PyTorch) ? multi-sport strong baseline (badminton/tennis/volleyball/?), fits our `SportProfile` abstraction; needs a custom-video inference wrapper. https://github.com/nttcom/WASB-SBDT
- True turnkey commercial APIs (Roboflow hosted models) exist but cap below a tuned TrackNet; enterprise systems (Hawk-Eye, SkillCorner, TRACAB) are not API-accessible.

Decision

1. Adopt **self-hosted TrackNetV4** as the primary precise shuttle detector, with **WASB** as the multi-sport fallback.
2. Run these models under **WSL2 (Ubuntu)**, **not** native Ubuntu. The entire indexer already runs on Windows; WSL2 keeps a single dev environment and quarantines the TensorFlow/PyTorch dependencies. Native Ubuntu would require dual-boot and migrating the whole project for marginal gain.
3. Integrate via a **subprocess adapter**: a detector module that shells out to the WSL `predict.py`, reads the coordinate CSV, and converts trajectory ? candidate rallies feeding the existing stitching flow. This keeps the TF dependency fully isolated from the main Python environment.

Consequences

- **WSL I/O caveat:** `/mnt/c` access is slow. Stage match videos inside the WSL filesystem (e.g. `~/clips/`) before inference rather than reading large mp4s across the Windows?WSL boundary.
- TrackNetV4 (TensorFlow) and WASB (PyTorch) live in **separate conda envs**.
- Next step after integration: manual video annotation ? fine-tune the detector.

ADR-002: Use the sports-data-collector's rally annotations as a golden eval + temporal-segmenter training set

Date: 2026-05-26
Status: Accepted

Context

A sibling project, **sports-data-collector**, curates sports videos and stores per-rally `[start, end)` timestamps (plus stroke counts, scores) in an SQLite DB, with a commercial-license flag per source. We asked whether this data could improve the indexer's rally detection, and for which task.

Key findings:

- The annotations are **temporal-only** (when rallies start/end). They do **not** contain
  per-frame shuttle `(x, y)` coordinates, so they **cannot train the shuttle detector**
  (TrackNetV4/WASB ? see ADR-001), which needs coordinate labels.
- They are an excellent **golden evaluation set** and a **training signal for the
  temporal rally segmenter** (active vs. dead play over time) ? the stage that is today
  a hand-tuned heuristic, not a learned model.
- The collector's five sports tables (`badminton_rallies`, `tennis_points`,
  `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`) all share a
  `(video_id, ordinal, start_time, end_time)` shape, and the indexer already has
  `sport_registry` / `segmenter_registry`. So a *generic* `(start, end)` bridge serves
  every sport.
- The two repos key videos differently: the collector by a human `video_id`, the indexer
  by **file MD5**. A re-encode/re-download changes the bytes, hence the MD5.

Decision

1. Treat the collector's rally timestamps as (a) the **golden eval set** for rally
   segmentation and (b) the **labels** for training a temporal segmenter ? explicitly
   not as shuttle-detector training data.
2. Bridge the repos with a sport-agnostic **export** (`curate export` in the collector):
   per-video `start,end` CSVs (the indexer's eval format) + a `manifest.json` that pairs
   each CSV with its video file. Gate on `is_commercial` so only commercially usable data
   flows through.
3. Honour the **MD5 keying** contract: evaluation/processing must run on the exact video
   file recorded in the manifest; acquisition (ADR-003) therefore precedes processing.

Consequences

- The collector becomes the *producer* of labels; the indexer the *consumer* (processing +
  scoring + training). Neither takes a hard dependency on the other beyond the manifest.
- Scoring is free and deterministic (reads the DB; no API). See `docs/EVALUATION.md`.
- Only invest in frame-level coordinate labels (for the detector) **if** evaluation shows
  detection, not *segmentation*, is the bottleneck.

ADR-003: Video acquisition ? always best resolution, upgrade in place, authenticated cookies

Date: 2026-05-26
Status: Accepted

Context

The collector's initial crawl downloaded videos with `yt-dlp -f worstvideo` (256×144),
mislabelled as 1080p. The indexer needs ?720p MP4 for meaningful YOLO/shuttle detection.
Re-acquiring full-resolution sources surfaced several hard-won realities:

- The ShuttleSet corpus **tops out at 1080p** (no 4K); "best available" resolves to 1080p.
- Authenticating with a **YouTube Premium** account is required for reliable, unthrottled
  downloads ? anonymous downloads are throttled (~1.3 MiB/s) and **non-deterministic**
  (identical commands variously yielded 1080p DASH, 1080p HLS, or a 144p fallback).
- **Chrome** cookies can't be read on Windows (app-bound encryption / DPAPI, yt-dlp
  #10927); **Firefox** cookies read cleanly even while open.
- The `web`/`web_safari` player clients are forced into **SABR streaming**, which strips

```
  download URLs and silently drops to 144p (yt-dlp #12482). yt-dlp's `default` client
  serves real downloadable 1080p DASH with auth cookies.
- Bulk bursts trip YouTube **rate-limiting** (a 22-video burst failed all at once, though
  each worked in isolation).

### Decision

1. **Always fetch the best resolution available** (`bv*+ba/b -S res,...`, merged to MP4);
   ffprobe the result and store the *actual* width/height/fps ? never assume.
2. **Upgrade in place, never clean-slate:** the `fetch` command re-downloads a video and
   updates only `local_path`/`resolution`/`fps`, preserving every rally annotation and
   curation status (deleting would cascade-delete the valuable labels).
3. Use **authenticated cookies** (`--cookies-from-browser firefox` / `--cookies file`) and
   the **`default` player client**; enable the EJS challenge solver (deno).
4. **Throttle bulk runs** (`--sleep` + per-video `--retries`/backoff) and treat a
   below-min-height download as a **failure** (discard, don't commit) so a flaky low-res
   result never overwrites a good entry.

### Consequences

- All 22 fetchable ShuttleSet matches are now true 1080p (one match, shuttleset_12, has no
  source URL in the catalog and remains video-less but keeps its annotations).
- Acquisition is reproducible and resumable; the ~26 GB corpus is kept outside git.
- Future non-YouTube or 4K sources are supported by the same code (optional
`max_height`).

---

## ADR-004: Add an offline, API-free *learned* rally segmenter as a first-class mode

**Date:** 2026-05-26
**Status:** Proposed (substrate pending data)

### Context

The default high-quality path (`yolo_hybrid`) refines YOLO candidate windows with **Gemini**,
which costs money and requires network access. With a now-1080p, commercially-licensed,
rally-annotated corpus (ADR-002/003), we can learn the refinement step offline instead.

Crucially, the **API-free signal already exists and is already persisted**: `yolo_hybrid`
Stage 1 is pure YOLO (player tracking ? per-window motion features: peak/mean velocity,
active-frame count, track-id count) stored in `candidate_segments.stats`. Gemini is only
the Stage-2 refiner. So the rally annotations can **train a classifier to replace Gemini**,
with no external API and **no new labeling**.

Three candidate substrates were weighed:

- **Tune `motion` thresholds** on the annotations ? trivial, fully offline, but a low
  ceiling (pixel motion can't separate rally from crowd/pan/replay). Use as a *floor*.
- **YOLO features ? trained classifier** ? reuses already-stored, semantically meaningful
  player-motion features; medium-high ceiling; moderate effort. *Recommended primary.*
- **WASB shuttle-motion ? rally detector** ? highest ceiling (the shuttle defines play),
  fully offline but heaviest (needs the WSL2 WASB runner, ADR-001); can't train WASB
  itself (no xy labels), only a rally layer on its output.

### Decision
```

```
179     1. Build an **offline, API-free learned segmenter** as a new registry mode (e.g.
180        `yolo_learned`): label detector windows by overlap with GT rallies, train a
lightweight
181        classifier on the persisted features, emit final segments with **zero API calls**,
and
182        score it with the existing harness against `motion` and a Gemini reference.
183     2. **Choose the substrate from data, not assumption.** A learned refiner can only
sharpen
184        the windows the detector found, so the deciding metric is the detector's **candidate-
stage
185        recall** vs. the annotations: high recall ? build on YOLO features; low recall ?
bring in
186        WASB's shuttle signal to raise the ceiling.
187     3. Keep the Gemini-backed `yolo_hybrid` as an **upper-bound reference**, not a
production
188        dependency.
189
190     ### Consequences
191
192     - The final tool gains a **self-contained, offline, free** rally mode trained on its own
193       curated data ? runnable air-gapped, improvable as more matches are annotated.
194     - The approach is **sport-generic**: the same label?feature?classifier machinery extends
to
195       tennis/cricket once those have annotations.
196     - The full paid 22-video Gemini evaluation is **not** run; spend is limited to a single
197       reference video.
198
199     ### Update (2026-05-26): Phase 3 evidence ? substrate is WASB, not YOLO
200
```

23. assistant (2026-06-21T14:37:15.170Z)

Perfect. Let me now check if there are any existing tests for storage backends to understand the test pattern:

24. user (2026-06-21T14:37:16.892Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_storage.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/tests/__pycache__/test_storage.cpython-313-pytest-9.0.3.pyc
```

25. user (2026-06-21T14:37:19.065Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       The S3/GCS backends are exercised with injected FAKE clients (no boto3/google-cloud-
storage,
4       no network); the gdrive backend reads a locally MOUNTED Drive, exercised against a
tmp_path
5       that stands in for the mount ? all offline, matching the repo's ethos.
6       """
7       import os
8       import subprocess
9       import sys
10
11      import pytest
12
13      from backend import storage
14      from backend.storage import StorageBackend, fetch, get_storage, list_uris, parse_uri,
put
15      from backend.storage.local import LocalStorage
16      from backend.storage.s3 import S3Storage
17      from backend.storage.gcs import GCSStorage
18      from backend.storage.gdrive import GDriveStorage
```

```
19
20
21      # ------------------------------------------------------------ URI parsing
22
23      @pytest.mark.parametrize("uri,backend,bucket,key", [
24          ("/data/x.mp4", "local", "", "/data/x.mp4"),
25          ("rel/x.mp4", "local", "", "rel/x.mp4"),
26          ("C:/Users/a/x.mp4", "local", "", "C:/Users/a/x.mp4"),       # Windows path: no
':'
27          ("C:\\Users\\a\\x.mp4", "local", "", "C:\\Users\\a\\x.mp4"),
28          ("local:///abs/x.mp4", "local", "", "/abs/x.mp4"),
29          ("s3://rally/videos/m.mp4", "s3", "rally", "videos/m.mp4"),
30          ("gs://rally-eu/w/model.json", "gcs", "rally-eu", "w/model.json"),   # gs:// -> gcs
31          ("gcs://rally/x", "gcs", "rally", "x"),
32          ("gdrive://Sharing/clip.mp4", "gdrive", "", "Sharing/clip.mp4"),     # path under My
Drive (no bucket)
33      ])
34      def test_parse_uri(uri, backend, bucket, key):
35          ref = parse_uri(uri)
36          assert (ref.backend, ref.bucket, ref.key) == (backend, bucket, key)
37
38
39      def test_parse_uri_rejects_unknown_scheme():
40          with pytest.raises(ValueError):
41              parse_uri("ftp://host/x")
42
43
44      # ------------------------------------------------------------ local backend
45
46      def test_local_fetch_is_identity_passthrough(tmp_path):
47          src = tmp_path / "v.mp4"
48          src.write_bytes(b"abc")
49          # No dst => returns the source path unchanged (no copy): preserves today's
behaviour.
50          assert fetch(str(src)) == str(src)
51
52
53      def test_local_put_and_roundtrip(tmp_path):
54          src = tmp_path / "in.csv"
55          src.write_text("Frame,Visibility,X,Y\n")
56          dst_uri = str(tmp_path / "out" / "copy.csv")
57          put(str(src), dst_uri)
58          assert (tmp_path / "out" / "copy.csv").read_text() == src.read_text()
59
60
61      def test_local_exists_stat_list_delete(tmp_path):
62          be = LocalStorage()
63          (tmp_path / "a.txt").write_text("hello")
64          (tmp_path / "sub").mkdir()
65          (tmp_path / "sub" / "b.txt").write_text("hi")
66          assert be.exists(parse_uri(str(tmp_path / "a.txt")))
67          assert not be.exists(parse_uri(str(tmp_path / "nope.txt")))
68          assert be.stat(parse_uri(str(tmp_path / "a.txt"))).size == 5
69          listed = sorted(r.key for r in be.list(parse_uri(str(tmp_path))))
70          assert listed == sorted([str(tmp_path / "a.txt"), str(tmp_path / "sub" / "b.txt")])
71          be.delete(parse_uri(str(tmp_path / "a.txt")))
72          assert not (tmp_path / "a.txt").exists()
73
74
75      def test_local_signed_url_raises():
76          with pytest.raises(NotImplementedError):
77              LocalStorage().signed_url(parse_uri("/x"))
78
79
80      def test_list_uris_helper(tmp_path):
```

```
        (tmp_path / "x.txt").write_text("1")
        (tmp_path / "y.txt").write_text("2")
        uris = sorted(list_uris(str(tmp_path)))
        assert all(u.startswith("local://") for u in uris)
        assert len(uris) == 2


    # ------------------------------------------------------------ registry / get_storage

    def test_get_storage_defaults_to_local():
        assert isinstance(get_storage(), LocalStorage)
        assert isinstance(get_storage(config={"backend": "local"}), LocalStorage)


    def test_get_storage_unknown_backend_raises():
        with pytest.raises(ValueError):
            get_storage("azure")


    def test_gdrive_not_mounted_raises_with_remedy(monkeypatch):
        # The scheme parses; if Drive isn't mounted (no auto-detected/configured root)
construction
        # is a clear, actionable error ? install link + how to include the folder in syncing
+ the
        # mount-root override ? rather than a cryptic failure downstream.
        monkeypatch.delenv("GDRIVE_MOUNT_ROOT", raising=False)
        assert parse_uri("gdrive://Sharing/x.mp4").backend == "gdrive"
        with pytest.raises(RuntimeError) as ei:
            # An explicit, non-existent root forces the error regardless of any real local
mount.
            get_storage("gdrive", config={"gdrive": {"mount_root": "/no-such/Drive/My
Drive"}})
        msg = str(ei.value)
        assert "drive.google.com" in msg or "google.com/drive" in msg   # install pointer
        assert "GDRIVE_MOUNT_ROOT" in msg                               # how to point at
the mount
        assert "shortcut" in msg.lower() and "stream" in msg.lower()    # how to sync the
folder


    def test_gdrive_mounted_but_unsynced_path_raises_with_remedy(tmp_path):
        # Mount exists but the requested file isn't there yet (folder not synced) -> a
FileNotFoundError
        # that names the fix (include the folder in syncing) instead of a bare "No such
file".
        be = GDriveStorage(mount_root=str(_mount(tmp_path)))
        with pytest.raises(FileNotFoundError) as ei:
            be.fetch_to_local(parse_uri("gdrive://Sharing/not-synced.mp4"), str(tmp_path /
"out.mp4"))
        msg = str(ei.value).lower()
        assert "not present locally" in msg and "shortcut" in msg and "stream" in msg


    # ------------------------------------------------------------ S3 (fake client)

    class _FakeS3Client:
        def __init__(self):
            self.store = {}

        def upload_file(self, src, bucket, key, ExtraArgs=None):
            with open(src, "rb") as f:
                self.store[(bucket, key)] = f.read()

        def download_file(self, bucket, key, dst):
            with open(dst, "wb") as f:
```

```
                f.write(self.store[(bucket, key)])

        def head_object(self, Bucket, Key):
            if (Bucket, Key) not in self.store:
                raise KeyError("404")
            return {"ContentLength": len(self.store[(Bucket, Key)]), "ETag": '"e"',
                    "ContentType": "video/mp4"}

        def get_paginator(self, _op):
            store = self.store

            class _P:
                def paginate(self, Bucket, Prefix=""):
                    yield {"Contents": [{"Key": k} for (b, k) in store
                                        if b == Bucket and k.startswith(Prefix)]}
            return _P()

        def delete_object(self, Bucket, Key):
            self.store.pop((Bucket, Key), None)

        def generate_presigned_url(self, op, Params, ExpiresIn):
            return f"https://s3.example/{Params['Bucket']}/{Params['Key']}?X-Amz-
Expires={ExpiresIn}"


    def test_s3_backend_roundtrip_with_fake_client(tmp_path):
        be = S3Storage(client=_FakeS3Client())
        assert isinstance(be, StorageBackend)
        src = tmp_path / "m.mp4"
        src.write_bytes(b"VIDEO")
        ref = parse_uri("s3://rally/videos/m.mp4")
        be.put(str(src), ref, content_type="video/mp4")
        assert be.exists(ref)
        assert be.stat(ref).size == 5
        dst = tmp_path / "dl" / "m.mp4"
        assert be.fetch_to_local(ref, str(dst)) == str(dst)
        assert dst.read_bytes() == b"VIDEO"
        assert [r.uri for r in be.list(parse_uri("s3://rally/videos/"))] ==
["s3://rally/videos/m.mp4"]
        url = be.signed_url(ref, ttl=900)
        assert url.startswith("https://s3.example/rally/videos/m.mp4") and "900" in url
        be.delete(ref)
        assert not be.exists(ref)


    def test_s3_exists_false_on_missing():
        assert S3Storage(client=_FakeS3Client()).exists(parse_uri("s3://b/none")) is False


    # ---------------------------------------------------------------- GCS (fake client)

    class _FakeBlob:
        def __init__(self, store, bucket, name):
            self.store, self.bucket, self.name = store, bucket, name
            self.size = self.updated = self.etag = self.content_type = None

        def upload_from_filename(self, src, content_type=None):
            with open(src, "rb") as f:
                self.store[(self.bucket, self.name)] = f.read()
            self.content_type = content_type

        def download_to_filename(self, dst):
            with open(dst, "wb") as f:
                f.write(self.store[(self.bucket, self.name)])

```

```
        def exists(self):
            return (self.bucket, self.name) in self.store

        def reload(self):
            self.size = len(self.store[(self.bucket, self.name)])

        def delete(self):
            self.store.pop((self.bucket, self.name), None)

        def generate_signed_url(self, version, expiration, method):
            return f"https://gcs.example/{self.bucket}/{self.name}?version={version}"


    class _FakeGCSClient:
        def __init__(self):
            self.store = {}

        def bucket(self, name):
            return type("_B", (), {"blob": lambda _self, key: _FakeBlob(self.store, name,
key)})()

        def list_blobs(self, bucket, prefix=""):
            return [_FakeBlob(self.store, bucket, k) for (b, k) in self.store
                    if b == bucket and k.startswith(prefix)]


    def test_gcs_backend_roundtrip_with_fake_client(tmp_path):
        be = GCSStorage(client=_FakeGCSClient())
        src = tmp_path / "w.json"
        src.write_text("{}")
        ref = parse_uri("gcs://rally/weights/w.json")
        be.put(str(src), ref)
        assert be.exists(ref)
        assert be.stat(ref).size == 2
        dst = tmp_path / "dl" / "w.json"
        be.fetch_to_local(ref, str(dst))
        assert dst.read_text() == "{}"
        assert [r.uri for r in be.list(parse_uri("gcs://rally/weights/"))] ==
["gcs://rally/weights/w.json"]
        assert be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
        be.delete(ref)
        assert not be.exists(ref)


    # ------------------------------------------------------------- Google Drive (local
mount)
    # The backend reads a locally MOUNTED Google Drive (Google Drive for Desktop). A
tmp_path
    # folder stands in for the mount's "My Drive" root ? no network, no Drive API, no extra
deps.

    def _mount(tmp_path):
        root = tmp_path / "My Drive"
        (root / "Sharing").mkdir(parents=True)
        return root


    def test_gdrive_backend_roundtrip_over_local_mount(tmp_path):
        root = _mount(tmp_path)
        be = GDriveStorage(mount_root=str(root))
        assert isinstance(be, StorageBackend)
        # A file present in the mount, exactly as Drive for Desktop would surface it.
        (root / "Sharing" / "clip.mp4").write_bytes(b"VIDEO")
        ref = parse_uri("gdrive://Sharing/clip.mp4")
        assert be.exists(ref)
```

```
260         assert be.stat(ref).size == 5
261         # No dst => identity passthrough to the on-mount file (no needless copy).
262         assert os.path.samefile(be.fetch_to_local(ref), root / "Sharing" / "clip.mp4")
263         # A dst => copy out (byte-stable scratch copy), like the local backend.
264         dst = tmp_path / "scratch" / "in.mp4"
265         assert be.fetch_to_local(ref, str(dst)) == str(dst)
266         assert dst.read_bytes() == b"VIDEO"
267         # list a folder -> gdrive:// URIs relative to the mount; `name` is the basename.
268         assert [r.uri for r in be.list(parse_uri("gdrive://Sharing"))] ==
["gdrive://Sharing/clip.mp4"]
269         assert [r.name for r in be.list(parse_uri("gdrive://Sharing"))] == ["clip.mp4"]
270         be.delete(ref)
271         assert not be.exists(ref)
272
273
274     def test_gdrive_put_writes_into_the_mount(tmp_path):
275         root = _mount(tmp_path)
276         src = tmp_path / "out.mp4"
277         src.write_bytes(b"REEL")
278         ret = GDriveStorage(mount_root=str(root)).put(str(src),
parse_uri("gdrive://Sharing/out.mp4"))
279         assert ret.uri == "gdrive://Sharing/out.mp4"
280         assert (root / "Sharing" / "out.mp4").read_bytes() == b"REEL"
281
282
283     def test_gdrive_mount_root_from_env(tmp_path, monkeypatch):
284         root = _mount(tmp_path)
285         (root / "v.mp4").write_bytes(b"X")
286         monkeypatch.setenv("GDRIVE_MOUNT_ROOT", str(root))
287         assert GDriveStorage().exists(parse_uri("gdrive://v.mp4"))         # env supplies
the root
288
289
290     def test_gdrive_autodetect_is_safe_across_platforms():
291         # Platform-dependent (a real mount or None), so assert only the invariant: it never
crashes
292         # and any hit ends at a 'My Drive' folder. Covers the detect logic on both win/posix
lanes.
293         from backend.storage.gdrive import _autodetect_mount_root
294         root = _autodetect_mount_root()
295         assert root is None or root.endswith("My Drive")
296
297
298     def test_gdrive_exists_false_on_missing(tmp_path):
299         assert
GDriveStorage(mount_root=str(_mount(tmp_path))).exists(parse_uri("gdrive://nope.mp4")) is False
300
301
302     def test_gdrive_signed_url_not_supported(tmp_path):
303         # A mounted Drive file has no presigned URL; callers fall back to fetch_to_local.
304         with pytest.raises(NotImplementedError):
305
GDriveStorage(mount_root=str(_mount(tmp_path))).signed_url(parse_uri("gdrive://v.mp4"))
306
307
308     # ------------------------------------------------------------- lazy-import hygiene
309
310     def test_importing_storage_does_not_pull_cloud_sdks():
311         # `import backend.storage` must not eagerly import boto3 / google-cloud-storage, so
the
312         # default install (and the CI lanes without those extras) stays clean.
313         code = (
314             "import sys, backend.storage; "
315             "assert 'boto3' not in sys.modules, 'boto3 imported eagerly'; "
316             "assert 'google.cloud.storage' not in sys.modules, 'gcs imported eagerly'; "
```

```
317             "print('clean')"
318         )
319         r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
320         assert r.returncode == 0, r.stderr
321         assert "clean" in r.stdout
322
323
324     def test_config_storage_defaults_are_non_breaking():
325         from backend.config import StorageConfig
326         sc = StorageConfig()
327         assert sc.backend == "local" and sc.output_root == ""
328         assert isinstance(storage.get_storage(config=sc.model_dump()), LocalStorage)
329
```

26. assistant (2026-06-21T14:37:21.228Z)

Excellent. Now let me check the regex in ref.py to confirm hyphen support:

27. user (2026-06-21T14:37:22.204Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
```

28. assistant (2026-06-21T14:37:24.826Z)

Perfect! Now I have all the information needed. Let me compile my analysis.

29. user (2026-06-21T14:38:07.799Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
You sent (first 200 of 8364 bytes): {"summary": "M12 gdrive-api OAuth StorageBackend is a

Claude-doable backend implementation + DI fake + hermetic tests. The scheme regex already allows hyphens. The work is a straightforward mirror of
Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

30. assistant (2026-06-21T14:38:09.965Z)

Let me call StructuredOutput correctly with properly formatted JSON:

31. user (2026-06-21T14:38:36.382Z)

Structured output provided successfully

32. assistant (2026-06-21T14:39:02.347Z)

Perfect! Now let me provide the detailed findings as a final summary to the user:

M12 gdrive-api OAuth StorageBackend ? Scope & Implementation Plan

I've analyzed the badminton-highlight-indexer repository to scope M12 precisely. Here are the findings:

****The Plan (Claude-doable)****

1. **New Backend File:** `backend/storage/gdrive_api.py` ? `GDriveApiStorage(StorageBackend)` class
2. **URI Scheme:** `gdrive-api://<file_id>` (hyphen already supported by regex)
3. **Dependency Injection:** Constructor accepts injected `client` (fake for tests), or lazy-imports real Google Drive API
4. **Backwards Compat:** Existing `gdrive://` mount-only backend stays untouched
5. **No DB Migration:** Current schema is v6 (M8 cost-ledger); M9-consent reserves v7; M12 adds no schema changes

****Exact File:Line Edits Required****

backend/storage/ref.py
- **Line 21:** Regex `_SCHEME_RE` already supports hyphens ? ? pattern is `r"^([A-Za-z][A-Za-z0-9+.\-]*)://"`
- **Line 22:** Extend `_KNOWN` tuple from `("local", "s3", "gcs", "gdrive")` ? add `"gdrive-api"`

backend/storage/__init__.py
- **Lines 77?91:** Extend `_backend_class(name)` function:
  ```python
  if name == "gdrive-api":
      from backend.storage.gdrive_api import GDriveApiStorage
      return GDriveApiStorage
  ```

backend/config/models.py
- **Line 318:** Extend `StorageConfig.backend` Literal:
  - From: `Literal["local", "s3", "gcs", "gdrive"]`
  - To: `Literal["local", "s3", "gcs", "gdrive", "gdrive-api"]`
- **Line 329:** Extend `StorageConfig.input_backend` Literal (same change)
- **Lines 344?345:** Add `gdrive_api: dict[str, Any] = Field(default_factory=dict)` (parallel to existing `gdrive`)

****Dependency Injection Pattern (Mirror GCS)****

GCS Reference (backend/storage/gcs.py:18?25):
```python
```

```
def __init__(self, *, client: Any = None, project: Optional[str] = None, ...):
    if client is not None:
        self._client = client
        return
    from google.cloud import storage  # lazy
    self._client = storage.Client(project=project)
```

M12 GDriveApiStorage Pattern (backend/storage/gdrive_api.py):
```python
def __init__(self, *, client: Any = None, credentials: Optional[dict] = None, scopes:
Optional[list] = None, **_ignored):
    if client is not None:
        self._client = client  # injected fake for tests
        return
    # Real Drive API: lazy import + auth chain (env/ADC, never secrets in config)
    from google.auth.transport.requests import Request
    from google.auth import default as auth_default
    from googleapiclient.discovery import build
    creds, _ = auth_default(scopes=scopes or ["https://www.googleapis.com/auth/drive.readonly"])
    self._client = build("drive", "v3", credentials=creds)
```

****Existing `gdrive://` Backend ? Untouched****

- **File:** `backend/storage/gdrive.py` (lines 1?136)
- **How it differs:** Reads **locally-MOUNTED** Google Drive (via Drive for Desktop), no API calls, auto-detects mount root
- **Must coexist:** Both backends live side-by-side; existing code uses `gdrive://` mount-only, new code uses `gdrive-api://` (Drive API)

****Claude-doable vs. Owner-gated****

Item	Status	Notes
Backend class + scheme + DI fake	**Claude-doable**	~300 lines, mirrors gcs.py, no secrets
Hermetic unit tests (injected fake Drive)	**Claude-doable**	Fake client matching `tests/test_storage.py` density (~200 lines)
Config updates (_KNOWN, _backend_class, Literal)	**Claude-doable**	~10 lines, zero breaking changes
Real Google OAuth app (service account JSON)	**Owner-gated**	Google Cloud Console app + secrets (outside M12)
Real Drive API access (live credentials)	**Owner-gated**	`GOOGLE_APPLICATION_CREDENTIALS` env or workload identity (production only)
Database migration	**Not needed**	Current PRAGMA user_version = 6; M9-consent reserves v7

****Database Status****

- **Current version:** 6 (backend/infrastructure/database.py:110)
  - v1: raw candidate detections
  - v2: async job model
  - v3: execution policy
  - v4: personalization (user_id nullable)
  - v5: per-user model store
  - v6: per-job cost ledger (M8)
- **M9-consent requires:** v7 (separate PR, adds ToS/DPA/consent cols) ? reserves a bump
- **M12 requires:** **ZERO** schema changes (only backend logic, no table/column edits)

****Isolated PR Shape****

One coherent PR:
1. Add `backend/storage/gdrive_api.py` (GDriveApiStorage class, ~300 lines)

2. Extend `ref.py` _KNOWN + regex doc
3. Extend `__init__.py` _backend_class() + registry doc
4. Extend `config/models.py` StorageConfig Literals + `gdrive_api` dict
5. Add tests in `tests/test_storage.py` (~200 lines, mirrors S3/GCS fake pattern)
6. No migrations, no existing code changes

Effort: M (2?3 hours) ? straightforward backend plumbing, zero complexity, full test coverage.